



CYART

inquiry@cyart.io

www.cyart.io

AI-Context

Phase 1 – Week 1 Technical Documentation

Prepared by

AI Context Squad

Date

22 July 2026



Table of Contents

1. Introduction
2. Objective
3. Technology Stack
4. Environment Setup
5. Project Structure
6. API Implementation
7. Testing and Verification
8. Screenshots
9. Conclusion



Introduction

The AI Context project is designed to build an intelligent backend service capable of storing user conversations together with their semantic representations, known as vector embeddings.

During Week 1, the primary focus was to develop the backend infrastructure required for future AI capabilities. This involves FastAPI, setting up PostgreSQL database, AI-memory table using Neon with enabling pgvector extension and embedding walkthrough. An important phase in this week was embedding generation process. This captured the meaning of the text and stored in PostgreSQL using pgvector extension. This forms foundation for semantic search in future phases.

At the end of Week 1, the backend service successfully supports storing user conversations, generating vector embeddings and retrieving stored memory.

Objective

The objective of Week 1 was to establish the development environment required for the AI Context backend. This involved configuring FastAPI, PostgreSQL, project structure, and development tools to create a stable backend for implementing AI Context features. The project also aimed to integrate Sentence Transformers for generating text embedding and configure the pgvector extension in PostgreSQL to store vector responses of user.

This whole week established the backend service successfully by completing workflow of receiving user conversations, generating embedding scripts, storing them in PostgreSQL and storing the data through FastAPIs.



Technology Stack

Technology	Purpose
Python	Backend programming language
FastAPI	REST API framework
Uvicorn	ASGI server
PostgreSQL	Relational database
pgAdmin 4	Database administration
Visual Studio Code	Development environment
pgvector	Stores vector embeddings

The above table shows which technologies we have used in the Week 1 project.



Environment Setup

Before we began AI Context, I set up and configured the software. This was done so that we could all work in the same development environment and the project could run correctly.

Activities in Week 1:

1. We installed the backend python.
2. We set up VS Code to write and maintain the code of the project.
3. Created a Python virtual environment (.venv) for this project.
4. We installed FastAPI and Uvicorn using pip.
5. We created a project folder and put files inside like main.py, database.py, models.py, schemas.py, routers.py, init.py and .env, embedding.py.
6. We used PostgreSQL database using Neon.
7. We installed pgAdmin 4 for managing the database.
8. We created the database connection.
9. We used Uvicorn to run FastAPI on the server.
10. Lastly, we verified API using Swagger UI

Project Structure

The AI Context project follows a modular structure to improve code organization, readability, and maintainability. Each file has a responsibility, making it easier for team members to collaborate and extend the application in future development phases.



AEV_AI_CONTEXT

--app

- main.py
- database.py
- modules.py
- schemas.py
- routers.py
- __init__.py
- embedding.py

--.env

- requirements.txt
- README.md

File	Description
main.py	Defines FastAPI
database.py	Configures the database connection using SQL Alchemy.
modules.py	Defines the database table structure
schemas.py	Contains Pydantic models for request and response validation.
embedding.py	Generates 384-dimensional embeddings
.env	Stores database connection string and environment variables.



API Implementation

Method	Endpoint	Description
GET	/	Returns the welcome message.
GET	/health	Checks whether the API is running successfully.
POST	/memory	Stores user conversations together with vector embeddings.
GET	/memory	Retrieves all stored conversations from the database.

Sample request,

```
{  
  "user_id": "tanaya",  
  "conversation": "Hello, this is my first memory."  
}
```

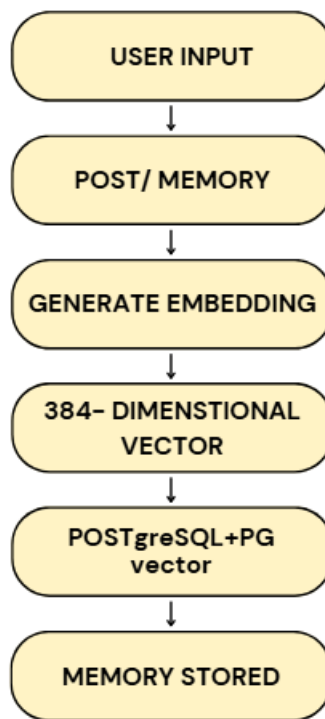


Sample response,

```
{  
  "message": "Memory and embedding saved successfully",  
  "id": 1,  
  "user_id": "tanaya",  
  "conversation": "Hello, this is my first memory.",  
  "embedding_dimension": 384  
}
```

Application Workflow

The overall workflow of the AI Context Project is shown below,



CYART

inquiry@cyart.io

www.cyart.io

Testing and Verification

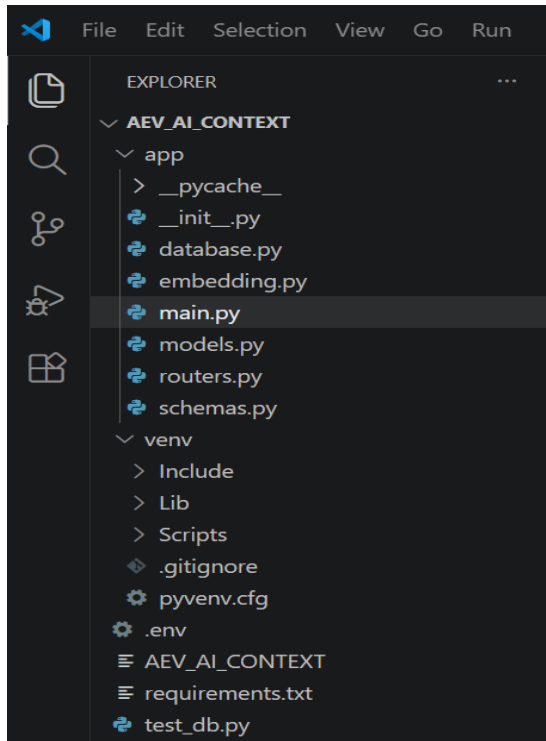
The application test was carried out using the Swagger Ui generated by FastAPI.

The functions which we successfully tested and verified are as follows:

1. FastAPI server setup
2. Database connectivity
3. PostgreSQL integration
4. Pgvector confirmation
5. Embedding
6. Memory storage using POST API
7. Memory retrieve using GET API
8. Successful storage of vector embeddings

Screenshots

1. AI Context project structure in Visual Studio Code

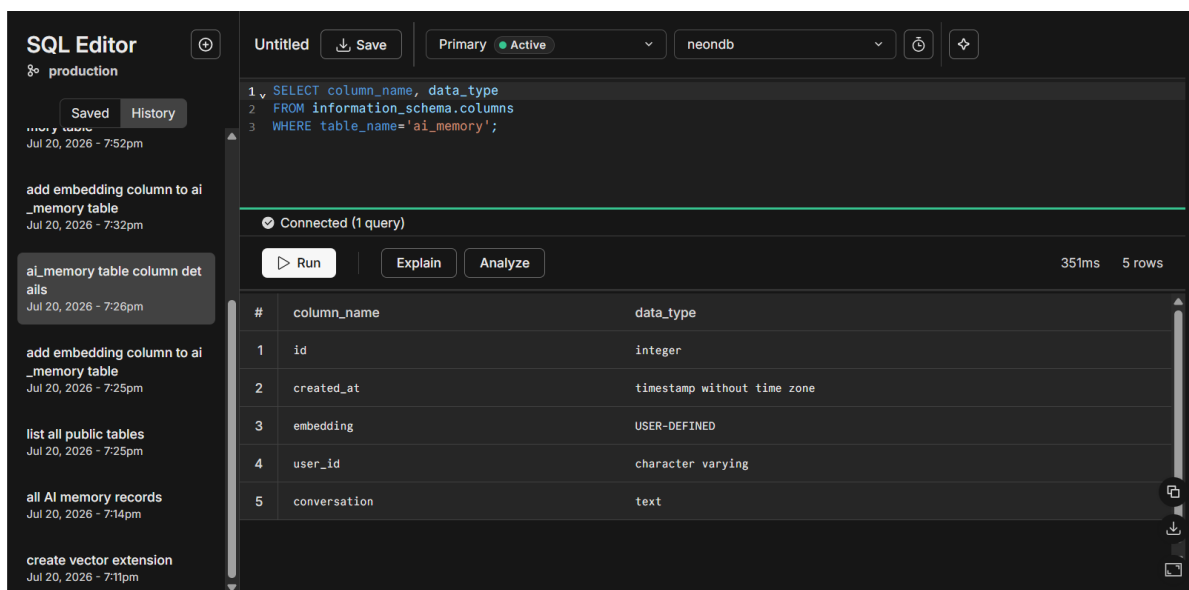


CYART

inquiry@cyart.io

www.cyart.io

2. Neon Ai memory table structure



3. Embedding script

```
app > embedding.py
1 from sentence_transformers import SentenceTransformer
2
3 model = SentenceTransformer("all-MiniLM-L6-v2")
4
5
6 def generate_embedding(text: str):
7     embedding = model.encode(text)
8     return embedding.tolist()
```

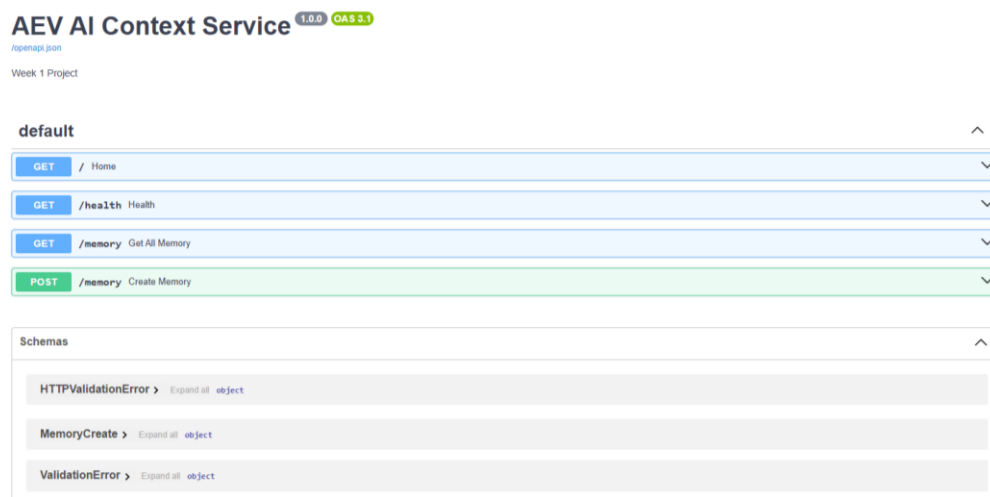


CYART

inquiry@cyart.io

www.cyart.io

4. Swagger UI displaying the implemented API endpoints.



5. Successful storage of conversation data and a 384-dimensional embedding.

POST

/memory

Create Memory

⌵

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

⌵

Edit Value

Schema

```
{
  "user_id": "tanaya",
  "conversation": "Hello, this is my first memory."
}
```

Execute

Clear

Responses



CYART

inquiry@cyart.io

www.cyart.io

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8001/memory' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "user_id": "tanaya",
    "conversation": "Hello, this is my first memory."
  }'
```

Request URL

http://127.0.0.1:8001/memory

Server response

Code

Details

200

Response body

```
{
  "message": "Memory and embedding saved successfully",
  "ip": " ",
  "user_id": "tanaya",
  "conversation": "Hello, this is my first memory.",
  "embedding_dimension": 384
}
```

Response headers

```
content-length: 154
content-type: application/json
date: Wed, 22 Jul 2026 06:57:17 GMT
server: uvicorn
```

Responses

Code

Description

Links

200

Memory saved successfully

No links

Media type

application/json

⌵

Controls

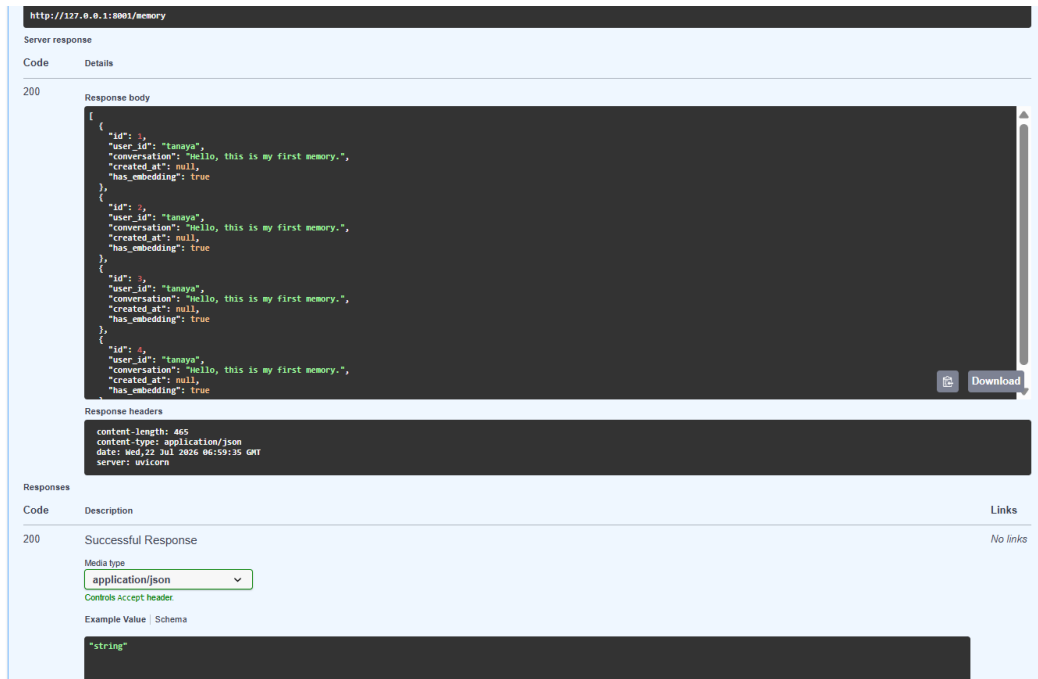
Accept header

Example Value

Schema

```
"string"
```

6. Successful GET /memory



inquiry@cyart.io

www.cyart.io

7. Terminal shows Uvicorn running on `http://127.0.0.1:8001`

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + v [ ] [ ] ... | [ ] [ ] [ ]
PS C:\Users\krtan\OneDrive\Desktop\AEV_AI_CONTEXT> python -m uvicorn app.main:app --reload --
port 8001
INFO: Will watch for changes in these directories: ['C:\\Users\\krtan\\OneDrive\\Desktop\\
\\AEV_AI_CONTEXT']
INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO: Started reloader process [15912] using StatReload
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to ena
ble higher rate limits and faster downloads.
Loading weights: 100%|████████████████████████████████████████████████████████████████████████████████| 103/103 [00:00<00:00, 563.35it/s]
INFO: Started server process [25772]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:54976 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:54976 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:59288 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:51594 - "GET /health HTTP/1.1" 200 OK

```

Conclusion

The implementation completed during Week 1 successfully verifies the backend foundation of AI Context project. FastAPI was configured as a framework and PostgreSQL was integrated as a database. User conversations are converted into 384-dimensional embeddings using all-MiniLM-L6-v2 Sentence Transformer model and the original data is also stored.

This project shows successful API development, database integration, embedding walkthrough and providing a scalable future AI-powered conversational memory.